Overview

This guide is for **frontend developers** wiring a UI on top of the BizPlay RAG Chatbot API. You'll find every endpoint a UI typically calls, the request/response shapes, and the end-to-end flows for the most common features (chat, bot management, document upload, **channel integrations** for Telegram and KakaoTalk).

- **Base URL**: `http://<base_url>/api/v1`
- **Authentication**: All `/api/**` endpoints are public — no `Authorization` header needed.
- **Content type**: `application/json` for all bodies except document upload (`multipart/form-data`).
- **Spec**: live OpenAPI / Swagger UI at `http://<base_url>/swagger-ui/index.html`.
- **Newly-created bots start disabled** — call `PATCH /api/v1/bots/{id}/enable` after the operator finishes uploading documents and tuning the system prompt. Disabled bots return `409` from chat but accept channel-link configuration so the operator can wire Telegram / KakaoTalk before going live.

Response envelope

Every JSON response is wrapped:

```
{
  "success": true,
  "message": "optional short human-readable note",
  "data": { /* endpoint-specific payload, may be null */ }
}
```

On error, `success: false`, `message` carries the explanation, and `data` is null:

```
{ "success": false, "message": "Bot is disabled", "data": null }
```

HTTP status reflects the outcome (`200` happy path, `400` bad request, `404` not found, `409` conflict, `500` server error). UIs should switch on HTTP status first, then read `message` for the user-facing string.

Core concepts

Concept	Notes
Corporation	The tenant that owns bots. Identified by `corp_no` (the natural business code issued by the external login service) — a soft reference, so the API stores the value but does not require a matching local row. `Default Corporation` (`corp_no = "DEFAULT"`) is seeded for the default-tenant fallback when no value is supplied. The local `corp` and `corp_group` rows are managed via `/api/v1/corps` and `/api/v1/corp-groups`.
Bot	The unit the user interacts with. Every bot has a name, description, system prompt, LLM model, and behaviour settings (temperature, history-turns, top-K, etc.).

Concept	Notes
	Documents and chat sessions belong to exactly one bot.
Document	An uploaded file (PDF / DOCX / TXT) ingested into the bot's vector store. Chunks of the document are retrieved during chat — but only when chatting with that document's bot.
Chat session	A multi-turn conversation. Sessions are bot-scoped: a `sessionId` is meaningful only within its bot. Pass the same `sessionId` to subsequent chat calls so the model sees prior turns. Omit it on the first message — the response carries the new ID. Each session carries a `channel` (`"web"` / `"telegram"` / `"kakaotalk"`) so analytics can group conversations by their origin.
Recommended question	A short canned starter prompt attached to a bot, for chat UIs that want to surface "Try one of these…" buttons. On the Telegram channel they're surfaced as inline-keyboard buttons under the `/start` greeting.
Channel integration	A bot can be additionally linked to a Telegram bot (long-polling, outbound HTTPS only) and/or a KakaoTalk channel (webhook + async callback via Kakao i Openbuilder). Same RAG pipeline serves all transports; sessions stamp `channel` accordingly. See **Channel integrations** below.

Endpoint reference

> Every URL is relative to the base `/api/v1`.

Bots

Create bot

`POST /bots`

Request body (only `name` and `llmModel` are required):

```
{
  "corpNo": "ACME-001",
  "name": "Travel Expense Bot",
  "description": "Helps employees with the travel reimbursement policy",
  "contactEmail": "travel-support@bizplay.com",
  "contactPhone": "+82-2-1234-5678",
  "systemPrompt": null,
  "sourceExpose": true,
  "llmModel": "exaone-3.5-7.8b",
  "llmTemperature": 0,
  "maxAnswerLength": 1024,
  "historyTurns": 5,
  "topK": 5,
  "recommendedQuestions": [
    { "question": "When can I claim travel expenses?" },
```

```
    { "question": "What is the per-diem rate for international trips?" }
  ]
}
```

Field	Type	Required	Default	Notes
`corpNo`	String	No	`"DEFAULT"`	Tenant — natural business code (max 50 chars). **Soft reference** — corp data is owned by the external login service, so any value is accepted (no local existence check). Defaults to the configured default tenant when omitted.
`name`	String	**Yes**	—	Max 255 chars.
`description`	String	No	null	Free-form.
`contactEmail` / `contactPhone`	String	No	null	Pure metadata.
`systemPrompt`	String	No	static default	When omitted, a sensible default is applied server-side. To draft one tailored to the bot's purpose, call `POST /bots/generate-system-prompt` first.
`sourceExpose`	Boolean	No	`true`	When `false`, chat responses for this bot return `sources: []`.
`llmModel`	String	**Yes**	—	Must match a name from `GET /rag/chat/models`.
`llmTemperature`	Number	No	`0`	Range `0.0–1.0`.
`maxAnswerLength`	Integer	No	`1024`	LLM `maxTokens`. Range 64–8192.
`historyTurns`	Integer	No	`5`	Number of prior turns sent in the prompt. Range 0–20.
`topK`	Integer	No	`5`	Chunks retrieved per chat. Range 1–50.
`recommendedQuestions[]`	Array	No	`[]`	Inline seed.

Response: `BotResponse` (see *Get bot* below). **Note**: regardless of the request body, newly-created bots are always returned with `disabled: true`. The operator finishes setup (documents, prompt, channel links) and then calls `PATCH /bots/{id}/enable` to make the bot accept chat traffic.

List bots

`GET /bots` → returns `BotSummary[]` for **every bot in the system, regardless of `corp_no`** (including disabled bots). Use `GET /bots/by-corp/{corpNo}` to filter by a specific corporation.


```
{
  "success": true,
  "data": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "corpNo": "ACME-001",
      "name": "Travel Expense Bot",
      "description": "Helps employees with the travel reimbursement policy",
      "llmModel": "exaone-3.5-7.8b",
      "disabled": false,
      "sourceExpose": true,
      "createdAt": "2026-04-29T10:00:00",
      "updatedAt": "2026-04-29T10:00:00"
    }
  ]
}
```


`corpNo` and `sourceExpose` are included on the summary so admin / cross-tenant UIs can group or filter bots, and clients can hide source panels in the UI, without an extra `GET /bots/{id}` call.

List bots by corporation

`GET /bots/by-corp/{corpNo}` → returns the same `BotSummary[]` shape as `GET /bots`, but filtered to a single `corp_no`. Includes disabled bots. `corp_no` is a soft reference to data owned by the external login service, so unknown codes return an empty list rather than `404`.

Use this when an admin / cross-tenant view needs to surface every bot for a specific corporation rather than the full list.

Get bot details

`GET /bots/{id}` → full `BotResponse` including `recommendedQuestions[]` and channel-integration flags:


```
{
  "success": true,
  "data": {
    "id": "550e8400-...",
    "corpNo": "ACME-001",
    "name": "Travel Expense Bot",
    "description": "...",
    "contactEmail": "...",
    "contactPhone": "...",
    "systemPrompt": "You are a helpful assistant ...",
    "sourceExpose": true,
    "llmModel": "exaone-3.5-7.8b",
    "llmTemperature": 0.0,
    "maxAnswerLength": 1024,
```

```
      "historyTurns": 5,
      "topK": 5,
      "disabled": false,
      "telegramConfigured": true,
      "telegramBotUsername": "BizPlayTravelBot",
      "telegramConfiguredAt": "2026-05-12T14:30:00",
      "kakaoConfigured": false,
      "kakaoBotName": null,
      "kakaoConfiguredAt": null,
      "recommendedQuestions": [
        { "id": "uuid", "question": "When can I claim travel expenses?" }
      ],
      "createdAt": "2026-04-28T10:00:00",
      "updatedAt": "2026-04-28T10:00:00"
    }
  }
```

Field	Notes
`telegramConfigured`	True when the bot is linked to a Telegram bot. The token itself is **never** returned.
`telegramBotUsername`	Telegram-side handle (e.g. `"BizPlayBot"`); useful for building a `t.me/{username}` link. Null when not linked.
`kakaoConfigured`	True when the bot has a Kakao i Openbuilder Skill webhook configured.
`kakaoBotName`	Operator-supplied display label (e.g. `"출장규정봇"`). Kakao doesn't give us a canonical name. Null when not linked.
`kakaoConfiguredAt` / `telegramConfiguredAt`	When the link was created. Null when not linked.

The Kakao webhook URL — which contains the per-bot secret — is **not** in `BotResponse`. Fetch it on demand from `GET /bots/{id}/kakao` (see Channel integrations below).

Update bot

`PUT /bots/{id}` — **PATCH semantics**: every field is optional, omitted fields stay unchanged. `recommendedQuestions` is special:

`recommendedQuestions` value	Effect
`null` (or field omitted)	Existing list left untouched.
`[]` (empty array)	All recommended questions cleared.
`[{question:"…"}, …]`	Replaces the entire list atomically.

`corpNo` cannot be changed via update.

Disable / enable

`PATCH /bots/{id}/disable` — bot is preserved but new chats return `409 Conflict`. Documents and history stay. `PATCH /bots/{id}/enable` — undo.

Delete

`DELETE /bots/{id}` — **hard delete**. Removes the bot's vector chunks, on-disk files, and the `bots` row in one transaction (pure JDBC). DB-level `ON DELETE CASCADE` then wipes documents, chat sessions, chat messages, and recommended questions. Irreversible.

Generate a system prompt

`POST /bots/generate-system-prompt` — stateless utility. Does **not** create a bot.

```
{ "name": "Travel Expense Bot", "description": "Helps with travel reimbursement",
"llmModel": "exaone-3.5-7.8b" }
```

→

```
{
  "success": true,
  "data": {
    "systemPrompt": "You are a helpful assistant specialising in the company's
travel reimbursement policy. ...",
    "llmModel": "exaone-3.5-7.8b"
  }
}
```

The generator mirrors the description's language (Korean description → Korean prompt; English → English). Use it as a "Generate" button on a bot-creation form. Falls back to a static default prompt if the LLM call fails — clients always get something usable.

Bot statistics

`GET /bots/{id}/statistics` → aggregate counters for a single bot. Returns `404` if the bot does not exist.

```
{
  "success": true,
  "data": {
    "botId": "550e8400-...",
    "botName": "Travel Expense Bot",
    "documentCount": 12,
    "completedDocumentCount": 11,
    "processingDocumentCount": 1,
    "failedDocumentCount": 0,
    "chatSessionCount": 47,
    "messageCount": 286,
    "conversationTurnCount": 143,
```

```
    "recommendedQuestionCount": 4,
    "lastChatAt": "2026-04-28T15:42:11"
  }
}
```

Field	Type	Notes
`botId`	UUID	Echoes the path parameter for convenience.
`botName`	String	Denormalised so callers don't need a second `GET /bots/{id}` call.
`documentCount`	Long	Total documents owned by this bot (any embedding status).
`completedDocumentCount`	Long	Documents whose embedding pipeline finished successfully.
`processingDocumentCount`	Long	Documents currently being embedded.
`failedDocumentCount`	Long	Documents whose embedding failed (re-upload required).
`chatSessionCount`	Long	Distinct chat sessions ever opened against this bot.
`messageCount`	Long	Total persisted chat messages (user + assistant combined).
`conversationTurnCount`	Long	Number of user-role messages — one user prompt + its assistant reply counts as one turn.
`recommendedQuestionCount`	Long	Recommended starter questions configured on the bot.
`lastChatAt`	DateTime	Timestamp of the most recent chat message. `null` if the bot has never been chatted with.

All counters come from JPA count queries (no row loading), so the call stays cheap regardless of history size.

Optional time window

Pass `?windowDays={N}` to additionally receive metrics restricted to the last *N* days plus the same metrics for the previous comparable window. Drives the analytics dashboard's KPI cards with their "Up X% compared to the previous period" deltas.

```
GET /bots/{id}/statistics?windowDays=7
```

Adds these fields to the response (all `null` when `windowDays` is omitted):

Field	Type	Notes
`windowDays`	Integer	Echoes the requested window size.
`windowStart`	DateTime	Inclusive lower bound (≈ `windowEnd − windowDays`).
`windowEnd`	DateTime	Exclusive upper bound (≈ now at request time).

Field	Type	Notes
`windowConversationCount`	Long	Sessions started inside the window.
`previousWindowConversationCount`	Long	Sessions started inside `[windowStart - windowDays, windowStart)`. Compute the delta client-side: `(windowConversationCount - previousWindowConversationCount) / previousWindowConversationCount`.
`windowMessageCount`	Long	Messages persisted inside the window.
`previousWindowMessageCount`	Long	Same metric for the previous comparable window.
`windowTokenUsage`	Long	Sum of `input_tokens + output_tokens` from assistant messages in the window. `0` for windows where the LLM didn't return a usage block (vLLM-served models sometimes omit it). Drives the dashboard's "Token usage" KPI card.
`previousWindowTokenUsage`	Long	Same metric for the previous comparable window — lets the UI compute the delta on the token-usage card.
`channelDistribution`	`Map<String, Long>`	Session counts grouped by channel (`web`, `telegram`, `kakaotalk`, ...) inside the window. Empty map when no activity. Drives the "Channel distribution" pie.
`languageDistribution`	`Map<String, Long>`	User-message counts grouped by detected language (`ko`, `en`, ...) inside the window. Restricted to user messages so the assistant's echoed answer doesn't double-count. Drives the "Distribution of languages used" widget.

Daily activity series

`GET /bots/{id}/statistics/daily?windowDays=7` → array of one bucket per calendar day inside the window, **zero-filled** (days with no activity appear with `conversationCount: 0`, `messageCount: 0`) so the bar-chart UI can render every label without gap-filling. Returns `404` if the bot does not exist; `400` if `windowDays` is missing or non-positive.

```
{
  "success": true,
  "data": [
    { "date": "2026-04-23", "conversationCount": 5, "messageCount": 28 },
    { "date": "2026-04-24", "conversationCount": 3, "messageCount": 14 },
    { "date": "2026-04-25", "conversationCount": 0, "messageCount": 0 },
    { "date": "2026-04-26", "conversationCount": 9, "messageCount": 47 },
    { "date": "2026-04-27", "conversationCount": 12, "messageCount": 62 },
```

```
      { "date": "2026-04-28", "conversationCount": 15, "messageCount": 81 },
      { "date": "2026-04-29", "conversationCount": 18, "messageCount": 94 }
    ]
  }
```

Field	Type	Notes
`date`	LocalDate	Server-timezone calendar day. Use as the bar-chart x-axis label.
`conversationCount`	Long	Sessions whose `createdAt` falls within this day.
`messageCount`	Long	Messages (user + assistant) whose `createdAt` falls within this day.

Backed by two single-shot native aggregate queries (`GROUP BY DATE(created_at)`) merged in the service — efficient regardless of how many days are in the window.

Popular keywords

`GET /bots/{id}/statistics/keywords?windowDays=7&limit=10` → top-`limit` most frequent keywords across user messages whose `createdAt` ∈ [now − windowDays, now). Returns `404` if the bot does not exist; `400` if `windowDays ≤ 0` or `limit ∉ [1, 100]`. `limit` defaults to `10`.

```
{
  "success": true,
  "data": [
    { "keyword": "출장",   "count": 32 },
    { "keyword": "비용",   "count": 28 },
    { "keyword": "항공권", "count": 21 },
    { "keyword": "숙박",   "count": 18 },
    { "keyword": "rental", "count": 14 }
  ]
}
```

Field	Type	Notes
`keyword`	String	A 1–3 word noun phrase. Multi-word phrases ("business trip", "travel expense") are produced naturally because the extractor runs an LLM topic-extraction pass over the user-message corpus rather than counting whitespace tokens. Returned in whichever language predominates in the input — Korean keywords for a Korean conversation, English for English.
`count`	Long	Estimated number of distinct user messages in the window that mention the keyword (or a synonym/inflection — e.g. `"비용을"`, `"비용이"`, `"비용은"` all roll up to `"비용"`).

How extraction works — one LLM call per request, using the bot's own configured chat model. The system prompt instructs the model to return only domain-specific topic phrases and to exclude generic verbs (`give`, `take`, `want`), courtesy words (`please`, `thanks`), pronouns, and conversational filler. The model tokenizes morphologically by understanding the language, which is why phrasing like "business trip" emerges as one keyword rather than two unrelated tokens.

Failure modes — keyword extraction is a non-critical-path operation. If the LLM call times out, returns malformed JSON, or the bot's model isn't reachable, the endpoint returns `200` with an empty `data` array rather than `500`. The dashboard treats no-data and extraction-failed identically (empty chip widget).

Cost — one chat completion per dashboard load. The user-message corpus is concatenated and capped at ~30k characters (~7k tokens, well inside the 32k context window of EXAONE) before being sent to the model; oldest messages are truncated first when the cap is exceeded. If load on the LLM becomes a concern, wrap the service method in `@Cacheable` with a 5–15 minute TTL or precompute on a `@Scheduled` job into a `bot_keyword_stats` table.

List chat sessions for a bot

`GET /bots/{id}/sessions` → array of session summaries for the given bot, ordered by creation time (newest first). Returns `404` if the bot does not exist; returns `200` with an empty array if the bot exists but has no sessions yet. Use this to enumerate conversations in an admin / dashboard view; for a single session's full transcript, follow up with `GET /rag/chat/history/{sessionId}`.


```
{
  "success": true,
  "data": [
    {
      "sessionId": "550e8400-e29b-41d4-a716-446655440000",
      "botId": "0580b2d3-ef97-4566-b095-c66b37c3257b",
      "createdAt": "2026-04-29T15:42:11",
      "lastMessageAt": "2026-04-29T15:48:33",
      "messageCount": 6
    }
  ]
}
```


Field	Type	Notes
`sessionId`	UUID	Pass to `GET /rag/chat/history/{sessionId}` to load the full transcript.
`botId`	UUID	Echoes the path parameter for convenience.
`createdAt`	DateTime	When the session was opened (typically equals the first user message timestamp).
`lastMessageAt`	DateTime	Most recent message timestamp. `null` if the session has no messages yet.
`messageCount`	Long	Total messages persisted for this session (user + assistant combined).

Backed by a single aggregate JPQL query — no N+1 even for bots with many sessions.

Recommended questions (incremental)

The full `recommendedQuestions[]` set can also be edited piece-by-piece without sending the full update payload:

Verb	Path	Body
GET	/bots/{id}/recommended-questions	—
POST	/bots/{id}/recommended-questions	{ "question": "..." }
DELETE	/bots/{id}/recommended-questions/{questionId}	—

The POST returns the persisted item (with a server-assigned UUID). Order is creation-time, ascending.

Tenant administration

CRUD for the local corp_group and corp rows. The local rows act as a directory and back the default-tenant fallback; bots only hold a soft reference (corp_no) to a corp row, so deleting a corp does not affect bots beyond leaving their corpNo value dangling.

Corp groups

POST /corp-groups — body { "corpGroupCd": "ACME-GRP" }. Returns:

```
{ "success": true, "data": { "id": 2, "corpGroupCd": "ACME-GRP" } }
```

Verb	Path	Notes
GET	/corp-groups	List all, ordered by corpGroupCd.
GET	/corp-groups/{id}	Path id is corp_group_id (BIGSERIAL).
PUT	/corp-groups/{id}	PATCH semantics; only corpGroupCd is updatable. 409 on collision.
DELETE	/corp-groups/{id}	409 if any corp row still references the group. Move/delete those corps first.

Corps

POST /corps — body { "corpNo": "ACME-001", "corpGroupId": 2, "corpName": "ACME Holdings" }. Returns:

```
{
  "success": true,
  "data": {
    "id": 7,
    "corpNo": "ACME-001",
    "corpGroupId": 2,
    "corpName": "ACME Holdings",
    "createdDate": "2026-04-29T10:00:00"
```

```
    }
  }
```

Verb	Path	Notes
`GET`	`/corps`	List all, ordered by `corpNo`. Optional `?corpGroupId=N` filter.
`GET`	`/corps/{corpNo}`	Path identifier is the natural business code.
`PUT`	`/corps/{corpNo}`	PATCH semantics; `corpGroupId` and `corpName` are updatable. `corpNo` is immutable — to change it, delete + re-create.
`DELETE`	`/corps/{corpNo}`	Removes the corp row. Bots that reference this `corp_no` keep their soft reference dangling — that's intentional.

Error	When
`400`	Validation failure (blank `corpGroupCd`, missing `corpName`, etc.)
`404`	`corp_group_id` (on corp create/update) or path id/`corpNo` not found
`409`	Duplicate `corpGroupCd` / `corpNo`, or trying to delete a non-empty `corp_group`

Documents

List models

`GET /rag/chat/models` → array used to populate the bot's "LLM model" dropdown:

```
{
  "success": true,
  "data": [
    { "name": "exaone-3.5-7.8b", "label": "EXAONE-3.5-7.8B", "isDefault": true },
    { "name": "gemma-4-8b",      "label": "Gemma-4-8B",      "isDefault": false }
  ]
}
```

Use the `name` value when sending `llmModel` on bot create/update.

Upload document

`POST /rag/documents/upload` — `multipart/form-data` with three parts:

Field	Type	Required	Description
`botId`	UUID	**Yes**	Bot the document belongs to. The bot must exist and not be disabled.
`file`	File	**Yes**	PDF, DOCX, or TXT. 50 MB maximum.
`title`	String	**Yes**	Display title used in chat sources.

Response (returns immediately; embedding may still be in progress):

```
{
  "success": true,
  "data": {
    "id": "doc-uuid",
    "botId": "bot-uuid",
    "title": "Travel Policy 2026",
    "fileName": "travel_policy_2026.pdf",
    "contentType": "application/pdf",
    "embeddingStatus": "PROCESSING",
    "createdAt": "2026-04-28T10:01:23"
  }
}
```

Poll `GET /rag/documents/{docId}` until `embeddingStatus` becomes `COMPLETED` (or `FAILED`).

List documents (per bot)

`GET /rag/documents?botId={botId}` — required query parameter. Returns the bot's documents, newest first.

Document download

`GET /rag/documents/{docId}/download` — returns the raw uploaded file inline with its original `Content-Type`. Suitable for `<a href>` "View source" links in chat UI.

Non-ASCII filenames (Korean, CJK, accented Latin) are handled via RFC 5987: the response carries a `Content-Disposition: inline; filename*=UTF-8''<percent-encoded>` header with an ASCII fallback. Modern browsers pick the UTF-8 form automatically; legacy clients see the ASCII fallback. No special handling needed on the frontend — just use `window.location.assign(documentUrl)` or a plain `<a href>`.

Delete document

`DELETE /rag/documents/{docId}` — removes the file on disk + every vector chunk for this document. Other documents on the same bot are unaffected.

Chat

Send a message

`POST /rag/chat`

```
{
  "botId": "550e8400-e29b-41d4-a716-446655440000",
  "query": "What's the daily allowance for an international trip?",
```

```
  "sessionId": null
}
```

Field	Type	Required	Notes
`botId`	UUID	**Yes**	Bot to ask. Returns `409` if disabled.
`query`	String	**Yes**	The user's question.
`sessionId`	UUID	No	Omit on the first message of a conversation. The response returns the newly-created `sessionId` — pass it back on subsequent calls. Sending a sessionId belonging to a different bot returns `400`.
`channel`	String	No	Free-form channel tag (max 20 chars; e.g. `"web"`, `"telegram"`, `"kakaotalk"`, `"slack"`). Persisted on the **session row at creation time** and ignored on subsequent messages of the same session — a session keeps the channel it was opened with. Defaults to `"web"` when omitted. Drives the dashboard's "Channel distribution" pie.

Response:

```
{
  "success": true,
  "data": {
    "answer": "The daily allowance for international trips is …",
    "sessionId": "session-uuid",
    "sources": [
      {
        "docId": "doc-uuid",
        "title": "Travel Policy 2026",
        "fileName": "travel_policy_2026.pdf",
        "snippet": "Article 5 (Per Diem) — international trips: $100/day…",
        "score": 0.87,
        "chunkIndex": 12,
        "documentUrl": "/api/v1/rag/documents/doc-uuid/download"
      }
    ]
  }
}
```

`sources` **is capped at one entry** — the single highest-relevance chunk. The retrieval pipeline still pulls `topK` chunks to populate the LLM's context window, but only the top result is exposed to the UI. This makes the citation UX simpler and prevents long lists from cluttering the chat bubble.

`sources` is an empty array when:

- the LLM responded with the refusal phrase `"I don't have enough information to answer this question."`
- or `bot.sourceExpose` is `false`

- or the vector search returned no chunks (bot has no documents, or none relevant to the query). In this case the answer is **still produced by the LLM**, but using the bot's system prompt + a "no documents available" directive — so operators with custom refusal phrasing (e.g. `"죄송합니다. 답변할 수 없는 질문입니다."`) and language rules get those honored automatically, in the user's language.

`score` is the reranker score when reranking is enabled (0–1, higher is better), otherwise `1 - cosineDistance`. Use it to colour-code source confidence.

Conversation history

`GET /rag/chat/history/{sessionId}` — returns every message in the session, oldest first:

```
{
  "success": true,
  "data": [
    { "role": "user",      "content": "What's the daily allowance?", "createdAt":
"..." },
    { "role": "assistant", "content": "The daily allowance is …",    "createdAt":
"..." }
  ]
}
```

Useful for restoring conversation state when the user reloads the page.

Channel integrations

A bot can be linked to a Telegram bot and/or a KakaoTalk channel. Both reuse the same RAG pipeline; the only difference is the transport that delivers user messages and bot replies. Configure each independently per bot; multiple bots can be linked at the same time, and a bot can have just Telegram, just Kakao, both, or neither.

Channel	Public ingress required?	Lifecycle	Multi-instance safe
Telegram	No — outbound long polling to `api.telegram.org`	One backend daemon thread per linked bot	No (Telegram 409s a second poller for the same token)
KakaoTalk	**Yes**, for `/api/v1/kakao/webhook/**` only — the rest of `/api/**` stays internal	Stateless POST + async callback	Yes

Disabled bots can be linked too — channel messages get a polite "currently unavailable" reply via the same channel until the bot is enabled. So the operator flow is: create bot → upload documents → tune prompt → link Telegram / Kakao → enable.

Telegram

The token is generated by @BotFather on Telegram. The frontend captures it, posts to our API, and we take over from there — polling thread, message routing, replies, and per-user session continuity all happen server-side.

`POST /bots/{id}/telegram` — link or replace a Telegram bot.

```
{ "token": "1234567890:ABCdefGHIjklMNOpqrSTUvwxYZ-1234567890" }
```

Response (token is never echoed back):

```
{
  "success": true,
  "data": {
    "telegramConfigured": true,
    "botUsername": "BizPlayTravelBot",
    "configuredAt": "2026-05-12T14:30:00"
  }
}
```

Errors: `400` invalid token (Telegram's `getMe` rejected it), `404` bot not found, `409` token already linked to a different bot in this system.

`GET /bots/{id}/telegram/status` — `{ telegramConfigured, botUsername, configuredAt }`. Token never returned.

`DELETE /bots/{id}/telegram` — unlink. Stops the poller thread, drops any residual webhook on Telegram's side, clears the row. Returns `{ telegramConfigured: false }`.

Operational notes:

- The integration runs **long polling** — our backend opens a long-lived HTTP call to `api.telegram.org` per linked bot. No webhook URL operators need to expose.
- **One instance only.** Telegram returns 409 on `getUpdates` if a second process polls the same token. For multi-replica deployments you need leader election before scaling out.
- **Private 1-on-1 chats only.** Group / supergroup / channel chats get a one-line "private only" reply.
- The Telegram link/unlink panel in the test UI also renders a **scannable QR code** of the `t.me/{username}` URL via the `qrcode-generator` library, so operators can hand the bot URL to users from their phone.
- Bot replies are sent as standalone messages (no `reply_to_message_id`) and split at 4000-char newline boundaries (Telegram caps each message at 4096 chars). Markdown from the LLM is converted to Telegram-friendly HTML.

KakaoTalk (via Kakao i Openbuilder)

Kakao only supports inbound webhooks. Operators set up a Skill in Kakao i Openbuilder and paste **our webhook URL** into the Skill config; messages then flow Kakao → our `/api/v1/kakao/webhook/{botId}/{secret}` → RAG pipeline → async callback back to Kakao.

`POST /bots/{id}/kakao` — link or **rotate the webhook URL**. Re-calling generates a fresh secret; the old URL stops working immediately.

```
{ "botName": "출장규정봇" }
```

Response includes the full webhook URL operators paste into Openbuilder (this is sensitive — the path contains the per-bot secret):

```
{
  "success": true,
  "data": {
    "kakaoConfigured": true,
    "botName": "출장규정봇",
    "webhookUrl":
"https://chat.example.com/api/v1/kakao/webhook/550e8400-.../9f7b1c8d4e6f0a2b1c8d4e
6f0a2b1c8d",
    "publicBaseUrlMissing": false,
    "configuredAt": "2026-05-12T14:30:00"
  }
}
```

Field	Notes
`webhookUrl`	The full URL including the per-bot secret. Treat as a credential — anyone with this URL can drive messages into the bot.
`publicBaseUrlMissing`	`true` if the server-side `KAKAO_PUBLIC_BASE_URL` env var isn't set, in which case `webhookUrl` is null. The UI should show a warning telling the operator to ask backend to configure the env var.

`GET /bots/{id}/kakao` — same shape, fetches the current status (use this to lazy-load the URL when the operator opens the bot editor, so it never sits in the bot list payload).

`DELETE /bots/{id}/kakao` — unlink. Clears the secret + display name + timestamp; the previously-issued webhook URL returns `404` immediately.

Operational notes:

- **Public ingress required**, scoped to `/api/v1/kakao/webhook/**` only. Configure your edge proxy (nginx / Cloudfront / etc.) to forward this path prefix to the internal app; the rest of `/api/**` stays internal.
- **5-second sync ACK + async callback.** Kakao expects a response within 5s but the LLM call takes 10–30s. The server replies synchronously with `useCallback: true` + a placeholder bubble (`"잠시만 기다려 주세요. 답변을 준비하고 있습니다…"`), then POSTs the real answer to `userRequest.callbackUrl` once the LLM finishes. Standard Openbuilder pattern.
- **Callback feature is gated by Kakao Business approval.** Until the *콜백 사용 (Use Callback)* feature is approved on the bot's Openbuilder account, real chat traffic will arrive without `callbackUrl` and our app replies with a "✓ webhook reachable" message instead of running the LLM. Apply via the *콜백 사용 신청* (Callback Usage Request) form in Openbuilder.

- **Multi-instance safe.** The webhook is a stateless POST; multiple app replicas behind a load balancer all work. The only per-turn state is the `callbackUrl` Kakao gives us — we use it once and forget.
- **Message rendering**: answers are split into up to 3 `simpleText` bubbles of 1000 characters each (Kakao's `template.outputs` × `simpleText.text` caps), split at paragraph / line / space boundaries.

Frontend setup flow (the test UI implements all of this, but a custom UI follows the same shape):

```
1. POST   /api/v1/bots/{id}/kakao  body { botName: "..." }
            → receive webhookUrl in response.

2. Display webhookUrl in a monospace block with a Copy button.
   Show a warning if response.publicBaseUrlMissing === true.

3. Operator copies the URL into Openbuilder's Skill config,
   enables "Use Callback", saves, and deploys the bot.

4. Operator sends a real KakaoTalk message; the user sees
   a placeholder bubble within 2s and the real answer
   10-30s later.

5. (When needed) POST again to rotate the URL,
   or DELETE to unlink entirely.
```

The webhook endpoint itself (`POST /api/v1/kakao/webhook/{botId}/{secret}`) is for Kakao to call — not for operators or the frontend. Path-secret auth: a wrong or missing secret returns `404` (collapsed with "bot not found" so attackers can't probe valid botIds).

End-to-end recipes

A. Build a chat UI for a single bot

```
1. GET    /api/v1/bots                          → pick a bot, store its id locally
2. GET    /api/v1/bots/{id}/recommended-questions → render as quick-pick chips
3. POST   /api/v1/rag/chat
            { botId, query, sessionId: null }    → first message; remember
response.sessionId
4. POST   /api/v1/rag/chat
            { botId, query, sessionId }          → subsequent messages, same
sessionId
```

Notes:

- Display `data.answer` with markdown rendering (the model emits markdown).
- `data.sources` should be presented as collapsible cards; honour the existing "I don't have enough information…" check (an empty `sources` array there is intentional).

B. Bot creation form

```
1. GET    /api/v1/rag/chat/models                → populate the LLM dropdown
2. POST   /api/v1/bots/generate-system-prompt
            { name, description, llmModel }      → optional "Generate" button
3. POST   /api/v1/bots
            { name, description, llmModel, systemPrompt, ...,
recommendedQuestions[] }
```

After create, the response carries the new bot's full configuration (incl. seeded recommended questions with their server-side UUIDs). No second `GET` is needed.

C. Document management

```
1. POST  /api/v1/rag/documents/upload  (multipart: botId + file + title)  → docId
2. Poll  GET /api/v1/rag/documents/{docId}  every ~3 s until embeddingStatus =
COMPLETED
3. List  GET /api/v1/rag/documents?botId={botId}
4. Delete DELETE /api/v1/rag/documents/{docId}
```

A bot's vector chunks become available for retrieval the moment `embeddingStatus = COMPLETED`. There's no "publish" step.

D. Restoring a conversation

```
1. Read sessionId from your client storage (e.g. localStorage)
2. GET   /api/v1/rag/chat/history/{sessionId}    → render messages
3. Continue with POST /api/v1/rag/chat using the same sessionId
```

If the user switches bot, drop the stored sessionId — sending it with a different botId returns `400`.

Error handling

HTTP	When	Typical UI action
`200`	Success	Render `data`
`400`	Validation failed (e.g. blank `query`, bad `temperature`, sessionId belongs to another bot, JSON parse error)	Show `message` near the offending input
`404`	Bot, document, or session not found	Toast / redirect
`409`	Bot is disabled, or the chat call hit a blocked state	Toast: "This bot is disabled"

HTTP	When	Typical UI action
`500`	Server-side bug	Generic "Something went wrong" toast; report `message` to the backend team

> Tip: a common `400` for new clients is **JSON parse error: Illegal unquoted character (CTRL-CHAR, code 10)** — multi-line strings (e.g. systemPrompt) need newlines escaped as `\n` inside JSON values.

Some notable points

1. **Sessions are bot-scoped.** Don't reuse `sessionId` across bots — start a fresh conversation when the user switches.
2. `POST /rag/chat` **doesn't take** `model` **/** `topK` **/** `exposeSources`**.** Those settings live on the bot. To change behaviour, update the bot via `PUT /bots/{id}`.
3. **Newly-created bots are disabled.** `POST /bots` returns the new bot with `disabled: true` regardless of what the request body said. Operators upload documents and tune the prompt, then call `PATCH /bots/{id}/enable`. Both Telegram and Kakao can be linked while the bot is still disabled.
4. **Documents need polling after upload.** `embeddingStatus` flips from `PROCESSING` → `COMPLETED` (or `FAILED`) only after the embedding pipeline finishes. Larger files (>5 MB) can take a couple of minutes. Non-ASCII filenames (Korean, etc.) survive the download endpoint via RFC 5987 encoding.
5. **Bot delete cascades hard.** Before calling `DELETE /bots/{id}`, confirm explicitly — vector chunks, files on disk, chat history, AND any Telegram / Kakao channel links all go. If the bot had Telegram linked, the deletion also best-effort calls Telegram's `deleteWebhook` to clear server-side state.
6. **Source list is capped at 1.** `data.sources` returns at most one entry regardless of the bot's `topK`. The cap is unconditional; `topK` only affects how many chunks the LLM sees internally.
7. **Refusal phrasing comes from the bot's system prompt.** Bots using the default system prompt return the exact English string `"I don't have enough information to answer this question."` when the LLM can't answer (use this for a string match if you need a custom refusal UI). Bots with a custom system prompt that defines its own refusal text (e.g. `"답변이 불가한 질문에 대해서는 '죄송합니다. 답변할 수 없는 질문입니다.' 라고 답변해주세요."`) will return *that* string instead, in the user's language. This applies both when the LLM can't answer from the available context AND when no documents are available at all.
8. **Default LLM behaviour is deterministic.** `llmTemperature = 0` means the same question yields the same answer. If your UX feels too rigid, raise temperature (max `1.0`) when creating/updating the bot.
9. **Source URLs are relative.** `documentUrl` returns `"/api/v1/rag/documents/{id}/download"` — prefix with the API base URL to hyperlink.
10. **Channel integration tokens are write-only.** `BotResponse` carries `telegramConfigured` / `kakaoConfigured` flags and non-secret display names, but never the Telegram bot token nor the Kakao webhook URL (which contains the per-bot secret). To get the Kakao webhook URL after the initial configure call, fetch `GET /bots/{id}/kakao`.